Automatic Question Generation from Study Material

Oct 28, 2026 · @Asil Khalifa

CSPE72 Deep Learning Techniques, Course Assignment Report. Team 3: Asil Khalifa (106123015), L Hitesh Reddy (106123073) and P Kamal Nag (106123093). Demonstration held on 29–30 October 2026; report submitted on 2 November 2026.

Abstract

Preparing practice questions from lecture material is a time-consuming task for instructors and learners. This report presents StudyQG, a system that generates verified, examination-style questions from study material supplied as a PDF file or as plain text. The system segments the material into passages, selects candidate answer spans with a combination of linguistic heuristics and a neural answer extractor, generates a question for each span with a T5 sequence-to-sequence Transformer fine-tuned on SQuAD v1.1, verifies every question through a round-trip check with an extractive question-answering model, and formats the verified questions as short-answer, multiple-choice, cloze and true/false items. A T5-small model fine-tuned by the team on 8,000 SQuAD examples in 49 minutes on a CPU attains BLEU-4 17.7, ROUGE-L 42.2 and METEOR 41.0 on 300 held-out paragraphs, against 2.7, 15.2 and 10.2 before fine-tuning; the publicly available T5-base checkpoint used in the demonstration attains 24.5, 49.7 and 49.5 under the same protocol. The complete system runs offline on a laptop without a GPU.

1. Team Details

Name	Roll No.	Email id	Contact number
Asil Khalifa	106123015	asilkhalifa11@gmail.com	
L Hitesh Reddy	106123073	hiteshreddy018@gmail.com	+91 63816 22523
P Kamal Nag	106123093	kamalnag1802@gmail.com	+91 70933 00985

Course: CSPE72 Deep Learning Techniques. Assigned problem statement: Automatic Question Generation from Study Material (Team 3).

2. Problem Statement

Instructors and learners spend considerable time composing practice questions by hand. The task addressed in this project is the automatic generation of examination-style questions from study material supplied in a common format: lecture notes as a PDF file, a textbook chapter as text, or pasted paragraphs. The system is required to produce well-formed, answerable questions that cover the material in four formats (short answer, multiple choice with plausible distractors, fill in the blank, and true/false), each accompanied by its answer and by the source sentence from which it was derived.

Formally, given a document D, the system produces a set of triples (q, a, t) such that the answer a is a span of D, the question q is answerable from D, the item type t is one of the four formats, the set covers the principal facts of D, and near-duplicate questions are removed. The central learning problem is answer-aware question generation: modelling $p(q \mid p, a)$ for a passage p and an answer span a with a sequence-to-sequence Transformer fine-tuned on the SQuAD dataset [1].

3. Gaps Identified

A review of existing approaches reveals five gaps between published question-generation research and a usable tool for study material.

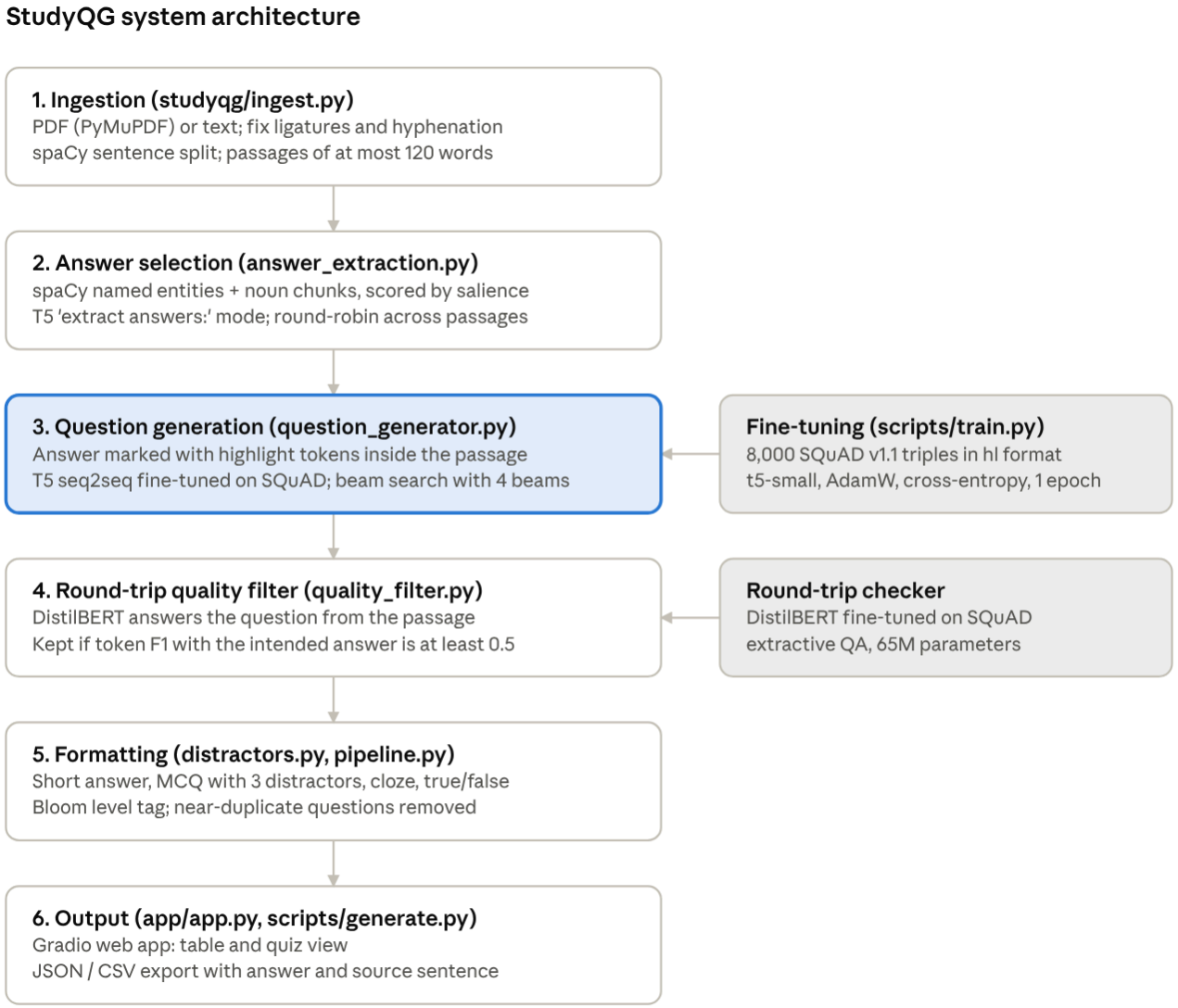
1. Rule-based systems. Early systems transform declarative sentences into questions with hand-written syntactic rules and rank the output statistically [2]. The resulting questions are often stilted, and the rules fail on the long, complex sentences that are typical of lecture notes.
2. Answer-aware models assume that the answer is given. Neural generators trained on SQuAD, from the attention-based LSTM of Du et al. [3] to pre-trained Transformers such as UniLM [4], ProphetNet [5] and the T5 models of Ushio et al. [6], generate a question for a provided answer span. Study material carries no marked answers, so a complete system requires answer selection and a strategy for covering the whole document.
3. Absence of an answerability check. Sequence-to-sequence generators can produce fluent questions that are unanswerable from the passage or that disclose the answer. Published pipelines rarely verify generated questions automatically; round-trip consistency has been used to filter synthetic training corpora [7] but not as an interactive quality gate for learners.
4. A single question format. Most systems produce only wh-questions, whereas assessment practice also requires multiple-choice items with plausible distractors, cloze items and true/false statements.

5. The clean-text assumption. Research code expects clean paragraphs, while real material arrives as PDF files with running headers, page numbers, hyphenated line breaks and typographic ligatures.

The system described in this report addresses each of these gaps: a T5 generator fine-tuned on SQuAD, a hybrid answer selector with document-level coverage, a round-trip verification stage, four item formats, and a PDF ingestion stage.

4. Proposed Architecture

The system is organised as a six-stage pipeline (Figure 1) built around two learned components: a T5 question generator fine-tuned on SQuAD, and a DistilBERT extractive question-answering model used for verification. The remaining stages are deterministic and serve to prepare the input for the generator and to package its output.



StudyQG architecture: 6 inference stages, 2 trained models

Figure 1. System architecture of StudyQG. The six stages on the left are executed for every input document; the two components on the right are the learned models.

Stage 1 (ingestion, module ingest.py) extracts text from PDF files with PyMuPDF, normalises ligatures and quotation marks, repairs hyphenated line breaks, discards lines that are predominantly non-alphabetic, splits the text into sentences with spaCy and groups sentences into passages of at most 120 words without crossing paragraph boundaries. Stage 2 (answer selection, answer_extraction.py) proposes candidate answer spans from named entities and noun chunks, scores them by entity type, syntactic role and document frequency, adds spans proposed by the T5 model in its answer-extraction mode, and selects candidates round-robin across passages so that the generated questions cover the whole document. Stage 3 (question generation, question_generator.py) marks the selected span with highlight tokens inside its passage and generates a question with beam search. Stage 4 (verification, quality_filter.py) submits each generated question together with its passage to the DistilBERT model and retains the question only if the token-level F1 between the predicted span and the intended answer is at least 0.5. Stage 5 (formatting, distractors.py and pipeline.py) produces the four item types, generates distractors, removes near-duplicates and assigns a Bloom taxonomy level. Stage 6 (output, app/app.py and scripts/generate.py) presents the items in a web interface and exports them as JSON and CSV. The question generator of stage 3 is the deep learning core of the system.

5. Algorithmic Steps

Algorithm 1. Fine-tuning of the question generator (scripts/train.py).

1. Load SQuAD v1.1 [1] and convert every (context, answer, question) triple into highlight format: the answer span is enclosed in `<hl>` tokens within the context and the task prefix "generate question:" is prepended. Sample 8,000 training and 300 validation examples with a fixed random seed.
2. Load the pre-trained t5-small model [8], register `<hl>` as an additional special token and resize the embedding matrix accordingly.
3. For each mini-batch of 8 examples, tokenise the source (at most 384 tokens) and the target question (at most 48 tokens); encode the source and decode the right-shifted target with teacher forcing; compute the token-level cross-entropy loss against the target.
4. Back-propagate the loss, clip the gradient norm to 1.0 and update the parameters with AdamW (learning rate 3e-4, weight decay 0.01) under a schedule of 5% linear warm-up followed by linear decay.
5. At the end of each epoch, evaluate the validation loss and save the model and tokenizer.

Algorithm 2. Question generation for a document (studyqg/pipeline.py).

1. Ingestion: read the PDF or text, normalise ligatures and quotation marks, repair hyphenated line breaks, discard predominantly non-alphabetic lines (page numbers, rules), split the text into sentences with spaCy, and group sentences into passages of at most 120 words without splitting a sentence or a paragraph.
2. Candidate answers: for every sentence, collect named entities (persons, dates, organisations, quantities) and noun chunks with spaCy; score each candidate by type and syntactic role and add a salience bonus proportional to the logarithm of its frequency in the document. When the lmqg checkpoint is used, also apply T5 in "extract answers:" mode to each sentence. Remove duplicates within a passage.
3. Selection: choose candidates round-robin across passages, one per sentence first, until 1.8 N candidates have been selected for a request of N questions, so that the questions cover the whole document.
4. Generation: highlight each selected answer within its passage and apply T5 with beam search (4 beams, no repeated trigrams, at most 64 new tokens). Normalise the output (capitalisation, terminal question mark), discard near-duplicates (string similarity above 0.9) and discard questions that contain the answer verbatim.
5. Verification: submit each question with its passage to the DistilBERT-SQuAD model; compute the token-level F1 between the predicted span and the intended answer; retain questions with F1 of at least 0.5, rank them by (F1, model confidence, candidate score), keep the top N and restore document order.
6. Formatting: cycle through the requested item types. Multiple choice: three distractors drawn from same-type spans of the document, WordNet co-hyponyms or perturbed numbers, shuffled with the answer. Fill in the blank: the source sentence with the answer removed. True/false: the source sentence unchanged, or with the answer replaced by a distractor. Assign a Bloom taxonomy level from the wording of the question.
7. Output: tabular view, quiz view, and JSON and CSV files containing the answer, quality score and source sentence of every item.

6. Dataset Used

Table 1. Datasets used in the project.

Dataset	Use	Size	Source
SQuAD v1.1 [1]	Fine-tuning and evaluation of the question generator	87,599 training and 10,570 validation triples from 536 Wikipedia articles; 8,000 training and 300 validation examples (one per paragraph) were sampled	rajpurkar/squad on the Hugging Face Hub
Sample study material	Demonstration inputs	Three passages of approximately 350 words written by the team on photosynthesis, neural networks and the Indian space programme	data/sample/ in the repository

SQuAD provides (context paragraph, question, answer span) triples. The task is reversed in this project: the model receives the context with the answer span highlighted and is trained to produce the question. The same dataset underlies the public lmqg checkpoints [6] and the DistilBERT question-answering model [9] used for verification, so every learned component shares one annotation convention.

7. Tools Used

Table 2. Software and hardware.

Tool	Version	Purpose
Python	3.12	Implementation language
PyTorch (CPU build)	2.14	Model training and inference
Hugging Face Transformers [12]	5.21	T5 and DistilBERT models, tokenizers, beam-search decoding
Hugging Face Datasets	5.1	Loading SQuAD v1.1
spaCy with en_core_web_sm [11]	3.8	Sentence segmentation, named entities, noun chunks
NLTK with WordNet	3.10	METEOR metric, WordNet distractors
sacrebleu, rouge-score	2.6	BLEU-4 and ROUGE-L metrics

Tool	Version	Purpose
PyMuPDF	1.27	PDF text extraction
Gradio	6.31	Web interface
pandas, matplotlib	3.1, 3.11	Result tables and plots
Git and GitHub		Version control and code submission
Claude Code (Anthropic)		AI coding assistant; see Section 14

All experiments, including fine-tuning, were executed on a laptop with a 16-core CPU, 15 GB of RAM and no GPU (Ubuntu under WSL2). PyTorch was restricted to 8 threads, which was measured to be substantially faster than 16 threads on this hardware.

8. Screenshots of the Result

Figures 2 and 3 show the web interface after generating ten questions from the neural-networks sample (398 words, four passages). Sixty-six candidate answers were extracted, eighteen questions were generated and all eighteen passed the round-trip verification; the mean quality score was 1.0. The reported wall-clock time of 157.6 s was measured while a training job shared the CPU. Figures 4 and 5 show the training curve of the fine-tuned model and the metric comparison discussed in Section 9.

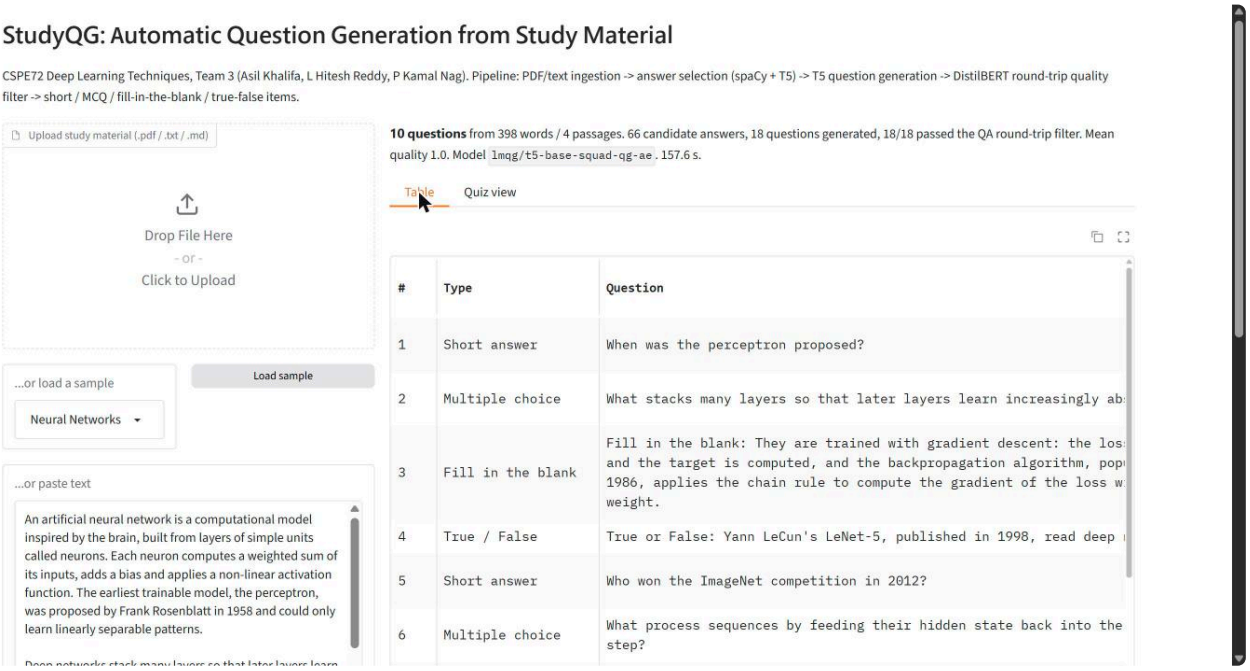


Figure 2. Tabular view of the generated questions in the web interface, with item type, question, options, answer, quality score, Bloom level and source sentence.

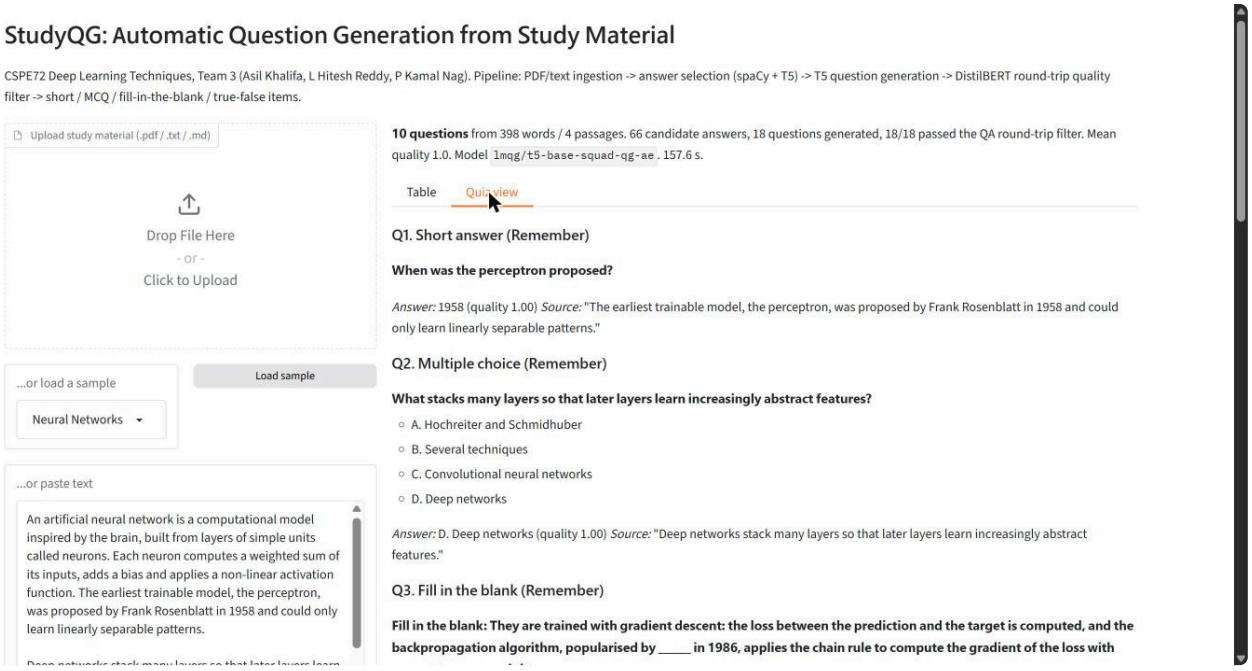


Figure 3. Quiz view with the answer key, the quality score and the source sentence of each item. Item Q4 is a true/false statement constructed by substituting a distractor for the verified answer.

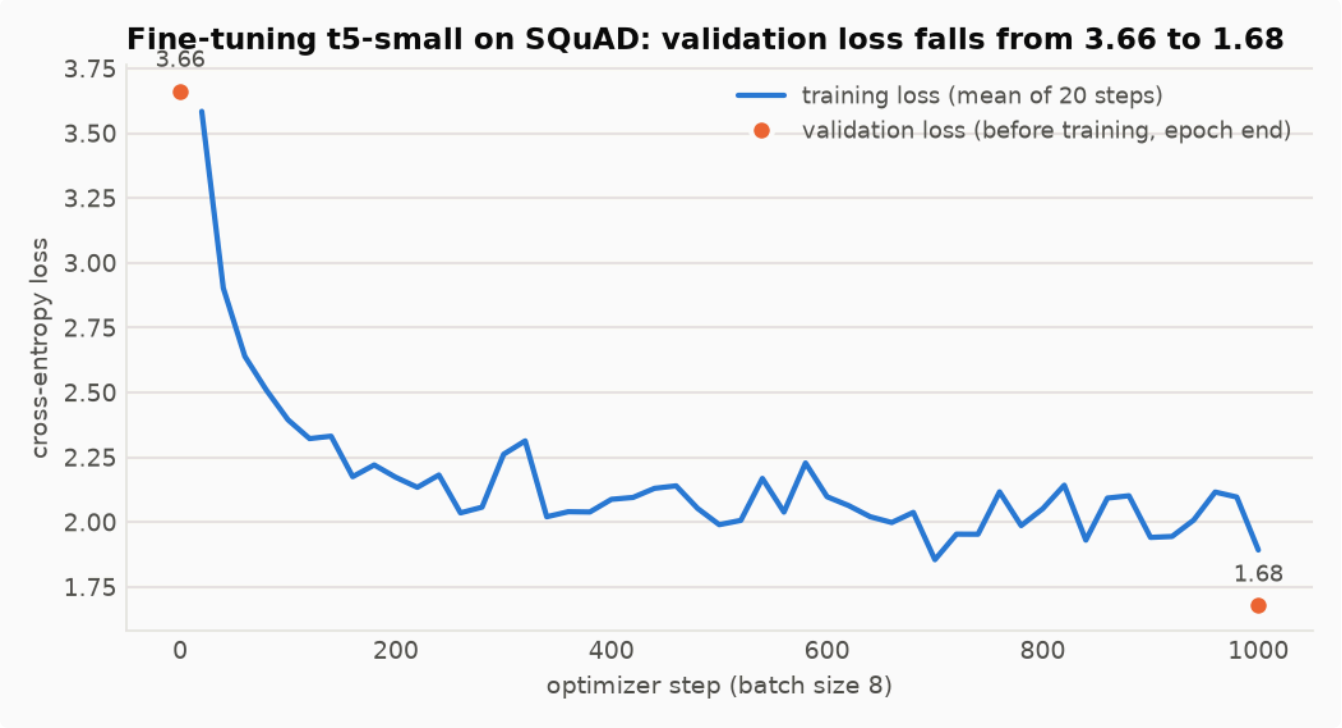


Figure 4. Training and validation cross-entropy loss during fine-tuning of t5-small on 8,000 SQuAD examples (1,000 optimiser steps, batch size 8, 49 minutes on the CPU). The validation loss decreases from 3.66 to 1.68.

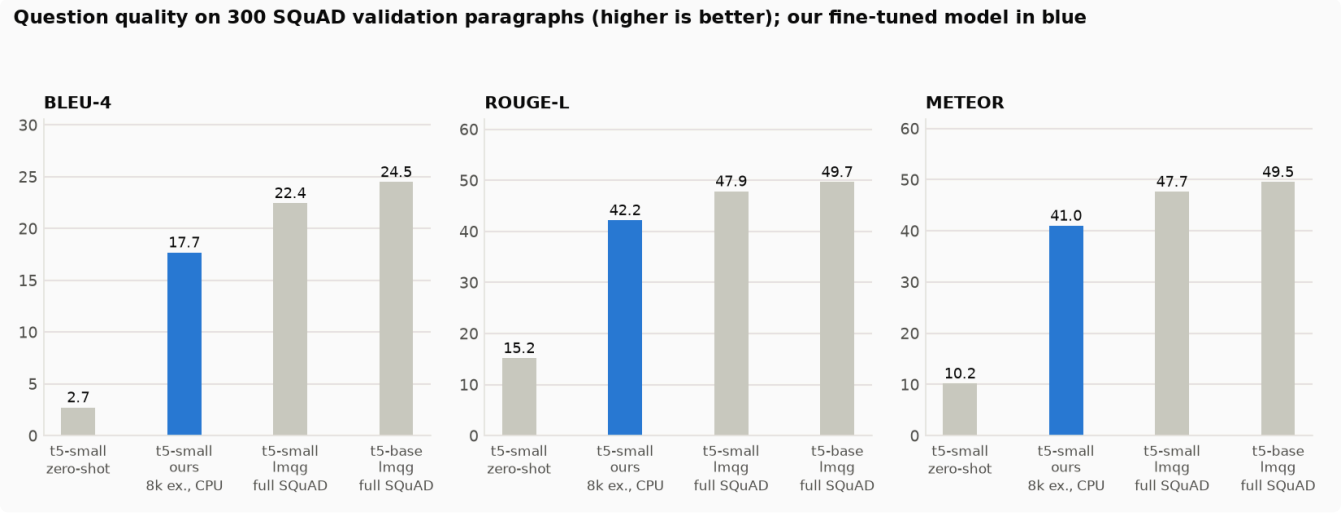


Figure 5. BLEU-4, ROUGE-L and METEOR of the four question generators on 300 SQuAD validation paragraphs. The model fine-tuned by the team is shown in blue.

9. Analysis and Comparison with Existing Approaches

9.1 Quantitative results

All generators were evaluated on 300 paragraphs of the SQuAD v1.1 validation set (one question per paragraph) with beam size 4, using sacrebleu for BLEU, rouge-score for ROUGE-L and the NLTK implementation of METEOR (scripts/evaluate.py). Scores range from 0 to 100 and higher values are better. Latency is the mean generation time per question on the laptop CPU with 8 threads.

Table 3. Evaluation of the question generators.

Model	Training data	BLEU-4	ROUGE-L	METEOR	Seconds per question
t5-small, no fine-tuning	none	2.7	15.2	10.2	0.55
t5-small, fine-tuned in this project	8,000 SQuAD examples, 1 epoch, 49 min on CPU	17.7	42.2	41.0	0.63
t5-small, lmqg [6]	full SQuAD (87,599 examples), GPU	22.5	47.9	47.7	0.91
t5-base, lmqg [6] (demonstration default)	full SQuAD (87,599 examples), GPU	24.5	49.7	49.5	2.69

Fine-tuning accounts for most of the attained quality. Without fine-tuning, T5 mostly copies a span of the passage instead of producing a question (for example, "the Teaching Council"). After 1,000 optimiser steps on a CPU, the validation loss decreases from 3.66 to

1.68 and BLEU-4 increases from 2.7 to 17.7, which closes approximately 80% of the gap to the lmqg t5-small checkpoint trained on eleven times more data with a GPU (76% on BLEU-4, 83% on ROUGE-L and 82% on METEOR).

Training data and model size account for the remainder. Training on the full dataset adds 4.8 BLEU-4 at the same model size, and the move from t5-small to t5-base adds a further 2.0 BLEU-4 and 1.8 ROUGE-L at three times the latency. The demonstration therefore uses t5-base by default and offers t5-small as a faster alternative.

Reference-based metrics underestimate question quality. Exact match with the reference question lies between 1% and 5% even for the strongest model, because a given answer admits many valid questions. For the answer "territory east of the Mississippi", the reference question is "What territory was ceded to Britain?", whereas the fine-tuned model produces "What territory did France cede to Great Britain?", a correct question with a low BLEU score. The round-trip verification stage addresses this limitation without a reference: on the neural-networks sample, 18 of 18 generated questions were answerable by DistilBERT with F1 of at least 0.5; on the ISRO sample, 11 of 15.

Two failure modes were observed. The small model occasionally produces an ungrammatical question (for example, "Who was the seniority of Genghis' oldest son Jochi?") or includes the answer in the question; the pipeline discards questions that contain the answer verbatim and relies on the verification stage for the remaining cases. Distractors for abstract noun phrases are weaker than those for names, dates and numbers.

9.2 Comparison with published question-generation models

Table 4. Published results on SQuAD compared with the models of this project.

System	Year	Architecture	Training data	BLEU-4	ROUGE-L	METEOR
Du et al. [3]	2017	BiLSTM encoder–decoder with attention	full SQuAD	12.3	39.8	16.6
Zhao et al. [10]	2018	Gated self-attention seq2seq with maxout pointer, paragraph level	full SQuAD	16.4	44.5	20.3
UniLM [4]	2019	Pre-trained unified Transformer	full SQuAD	22.1	51.1	25.1
ProphetNet [5]	2020	Pre-trained Transformer with n-stream decoder	full SQuAD	25.0	52.6	26.8
lmqg BART-base [6]	2022	BART encoder–decoder	full SQuAD	24.7	52.7	26.1

System	Year	Architecture	Training data	BLEU-4	ROUGE-L	METEOR
lmqg T5-small [6]	2022	T5 encoder–decoder, 60M parameters	full SQuAD	24.4	51.4	25.8
lmqg T5-base [6]	2022	T5 encoder–decoder, 220M parameters	full SQuAD	26.1	53.3	27.0
This project, t5-small fine-tuned on CPU	2026	T5 encoder–decoder, 60M parameters	8,000 SQuAD examples, 1 epoch	17.7	42.2	41.0

The published rows reproduce the figures reported in the respective papers and model cards, which use the test split of Du et al. [3] and the nlg-eval scorers; the row for this project uses 300 paragraphs of the official development set with sacrebleu and rouge-score. The comparison therefore indicates trends rather than exact values. Scoring the lmqg checkpoints under the protocol of this project (24.5 BLEU-4 and 49.7 ROUGE-L for T5-base, against 26.1 and 53.3 on the model card) quantifies the offset between the two protocols at approximately 2 to 4 points. METEOR values are not comparable across rows because the NLTK implementation credits stem and synonym matches. Under this reading, the model fine-tuned on a CPU in under one hour lies between the 2017 LSTM system [3] and the 2018 paragraph-level system [10] on BLEU-4 and ROUGE-L, while the T5-base checkpoint used in the demonstration is at the level of the strongest published SQuAD question generators.

9.3 Comparison of approach characteristics

Table 5. Characteristics of existing approaches and of the proposed system.

Approach	Requires a given answer	Verifies answerability	Item formats	Input handled
Rule-based transformation [2]	No; rules select trivial spans	Statistical ranking only	Wh-questions	Clean sentences
Seq2seq on SQuAD [3]	Yes	No	Wh-questions	Clean paragraphs
lmqg T5 checkpoints [6]	Yes, or a separate extractor	No	Wh-questions	Clean paragraphs

Approach	Requires a given answer	Verifies answerability	Item formats	Input handled
Prompting a hosted large language model	No	No; may hallucinate	Any	Pasted text; material leaves the device
StudyQG (this project)	No; hybrid answer selection with document coverage	Yes; DistilBERT round trip with a reported score	Short answer, multiple choice, cloze, true/false	PDF or text; offline on a CPU

9.4 Model selection

The task is the conditional generation of a short question from a long passage with one marked span. Encoder-only models such as BERT and DistilBERT [9] produce one representation per input token and cannot generate text; they are therefore used in this project for verification, where the output is a span of the input, rather than for generation. Decoder-only models of the GPT family can generate text but process the input strictly from left to right, so that the passage and the question compete for a single context window and the task must be conveyed through prompt wording. Encoder–decoder models encode the passage bidirectionally once and generate the question with a separate decoder that attends to every passage token through cross-attention; this family holds the published state of the art for SQuAD question generation [4, 5, 6].

Among encoder–decoder models, T5 [8] was selected for four reasons. First, its text-to-text formulation makes the task prefix and the `<hl>` highlight token a natural input representation and allows a single model to perform both answer extraction and question generation. Second, the model is available at sizes suited to the available hardware: t5-small (60 million parameters) was fine-tuned on a CPU in 49 minutes, and t5-base (220 million parameters) remains fast enough for a live demonstration. Third, public SQuAD checkpoints in exactly this input format [6] permit a controlled comparison between the model fine-tuned in this project and a full-data baseline. Fourth, pre-training on the C4 corpus provides strong transfer: 8,000 examples sufficed to raise BLEU-4 from 2.7 to 17.7. The trade-off is that a large hosted language model would produce more varied questions; the project prioritises a model that can be trained, measured and executed offline on the learner's own material.

10. Code Link

The complete source code, sample data, evaluation results, screenshots and this report are available at https://github.com/asil-khalifa/dl-project (public repository, branch main). The README documents installation, the demonstration and the reproduction of all experiments. The fine-tuned checkpoint is not stored in the repository; it is reproduced by scripts/train.py in approximately 49 minutes on a CPU.

11. Novel Contribution

Published question-generation models address the mapping from a passage and an answer to a question. The contribution of this project is the complete system that makes such a model usable on real study material, together with a generator fine-tuned by the team.

1. Round-trip verified generation. Every generated question is answered by an independent extractive question-answering model, and only questions whose recovered answer matches the intended answer (token F1 of at least 0.5) are retained. The round-trip consistency criterion of Alberti et al. [7], originally proposed for filtering synthetic training corpora, is thereby applied as an interactive quality gate, and each item carries a quality score visible to the user.
2. Coverage-aware hybrid answer selection. Candidates are obtained from two sources (spaCy entities and noun chunks with a document-frequency salience score, and the answer-extraction mode of T5) and are selected round-robin across passages, so that a ten-question quiz spans the whole document rather than its first paragraph.
3. Four item formats from one generator. Distractors are drawn from spans of the same type within the same document (a question on Chandrayaan-3 is offered other ISRO missions rather than unrelated terms), with WordNet and numeric fallbacks; cloze and true/false items reuse the verified source sentence.
4. A generator fine-tuned by the team and a controlled comparison. A t5-small model was fine-tuned on 8,000 SQuAD examples in 49 minutes on a CPU and evaluated against an untrained baseline and the full-data lmqg checkpoints with BLEU-4, ROUGE-L and METEOR, separating the contribution of fine-tuning from that of model size and training data.
5. End-to-end PDF handling with traceable output. Every item carries its source sentence and a Bloom taxonomy level, and the entire pipeline executes on a laptop without a GPU.

12. Individual Contribution

Member	Contribution
Asil Khalifa (106123015)	Pipeline design; T5 question generator and fine-tuning (question_generator.py, scripts/train.py); evaluation and plots; repository and README
L Hitesh Reddy (106123073)	Ingestion and answer selection (ingest.py, answer_extraction.py); sample study material; integration of the round-trip verification stage; unit tests
P Kamal Nag (106123093)	Distractor generation and item formatting (distractors.py, pipeline.py); web interface (app/app.py); report and demonstration

All members reviewed the code, executed the experiments and contributed to this report.

13. Learning Outcome

1. Fine-tuning a text-to-text Transformer for a new task: data formatting with task prefixes and highlight tokens, tokenisation, teacher forcing, cross-entropy loss, AdamW with warm-up, and checkpointing.
2. The distinction between training-time and inference-time behaviour of a sequence-to-sequence model, and the effect of decoding choices (beam size, repetition constraints) on the output.
3. Evaluation of generated text with BLEU-4, ROUGE-L and METEOR, the properties these metrics reward, and the need for a reference-free check such as round-trip question answering.
4. Use of pre-trained models from the Hugging Face Hub (lmqg T5, DistilBERT-SQuAD) as system components, and the trade-off between model size, training data and CPU latency.
5. The continued role of classical NLP tooling (spaCy named-entity recognition, noun chunks, WordNet) for answer selection and distractor generation around a neural model.
6. Engineering of a complete system: PDF ingestion, passage segmentation, batched CPU inference, a web interface, reproducible scripts and unit tests.

14. AI Tool Usage

Table 6. AI tools used and their purpose.

Tool	Purpose
Claude Code (Anthropic), model Claude Fable 5.1	Scaffolding of the repository and drafting of the pipeline code, training and evaluation scripts, web interface, README, this report and the team's study notes, under the direction of the team. The team reviewed, executed and debugged the code and verified every reported result. The prompts issued are listed in Section 15.
Pre-trained models from the Hugging Face Hub	google-t5/t5-small [8], the lmqg T5 question-generation checkpoints [6] and DistilBERT-SQuAD [9], used as components of the system.
spaCy and NLTK WordNet	Rule-based NLP components; not generative AI.

No AI tool was used to produce or alter results. All reported metrics originate from executions of scripts/evaluate.py on the SQuAD validation subset, and all screenshots were captured from the running application.

15. Prompts Issued to the AI Agent

The following prompts were issued by the team to the Claude Code agent in the project directory between 28 October and 1 November 2026. The agent had access to the assignment document, could execute code on the team's laptop and could edit this report. For each prompt, the text as issued is followed by a refined formulation that states the context, the constraints and the acceptance criterion explicitly; the refined formulations represent the prompting practice the team would adopt in future work.

1. Kick-off: build the whole submission

As issued: "do the project for our team (Team 3 asil hitesh and kamal) including all the things required to submit. Also create a cheatsheet document for us to explain things related to the project"

Refined: "You are helping Team 3 of CSPE72 Deep Learning Techniques (Asil Khalifa 106123015, L Hitesh Reddy 106123073, P Kamal Nag 106123093). Our assigned problem is Automatic Question Generation from Study Material; the assignment PDF in this folder

lists the deliverables (demo on 29–30 Oct, report and GitHub link on 2 Nov, 14 report sections). Build a complete, demo-able system: a deep learning model that we fine-tune ourselves plus a stronger public baseline, a web demo that accepts a PDF or pasted text, a GitHub-ready repository with README and tests, and a report covering every required section. Constraint: this laptop has 16 CPU cores and no GPU, so keep training under about an hour and report real measured numbers only. Also write a cheatsheet for the three of us that explains the concepts, the code and likely viva questions in plain language."

2. Divide the work among three members

As issued: "also split the tasks required for this project into 3 (for 3 members) in the cheatsheet - include stuff where we gave our (human) input (eg: research, ideation) and stuff where AI was more useful (eg: imlementation)"

Refined: "In the cheatsheet add a section that divides the project among the three members so that each owns one slice end to end. For every slice separate (a) the human contribution: literature reading, design decisions, threshold choices, testing on real notes, interpretation of results, and (b) the parts where the AI assistant did most of the implementation or drafting. Keep it honest and consistent with the Individual Contribution table in the report."

3. Progress check

As issued: "status?"

Refined: "Give a short status: what is finished, what is still running with an estimated finish time, what is blocked on us, and the links to the report and the cheatsheet."

4. Publish the code

As issued: "push here: https://github.com/asil-khalifa/dl-project"

Refined: "Push the repository to https://github.com/asil-khalifa/dl-project (public, branch main). Update the README clone URL and the report's Code link section to this address, and list what still remains for the submission."

5. Demo preparation

As issued: "how to setup the project for a demo"

Refined: "Write demo setup instructions for two cases: this laptop, where everything is installed, and a fresh laptop that clones the repo. Include pre-downloading the models, how to start the app, a recommended five-minute demo flow with timings, a faster fallback model, and what to show if the live demo fails."

6. Model choice

As issued: "what other techniques did you try apart from T5 transfomer? why did you choose this?"

Refined: "List the alternative model families you considered for question generation (rule-based, RNN seq2seq, BERT-style encoders, GPT-style decoders, BART, prompting a hosted LLM) and explain the trade-offs that led to T5. State clearly which alternatives were actually run and which were only considered."

7. Understanding the quality filter

As issued: "if bert is encoder only and can't generate text how did we use it to do QA check on the questions given by T5"

Refined: "Explain how an encoder-only model such as DistilBERT answers a question without generating text (the start and end span heads), how we compare its span with the intended answer, and give a one-line version we can say in the viva."

8. Understanding the pipeline order

As issued: "so when the doc is first given, T5 and spacy first pick out answers to question? and then T5 generates questions for those answers?"

Refined: "Confirm the order of the stages: how answer spans are chosen (spaCy and the T5 extract-answers mode), how T5 then generates a question for each span, and what changes when our own fine-tuned model is used instead of the public one."

9. Understanding the metrics

As issued: "what are those blue1 rouge l"

Refined: "Explain BLEU-1, BLEU-4, ROUGE-L, METEOR and exact match in one short paragraph each, with an example from our own outputs and our actual scores, and say what each metric cannot measure."

10. Understanding the Bloom tag

As issued: "which stage comes up with the bloom"

Refined: "Which pipeline stage assigns the Bloom level, how exactly is it computed, and how should we describe its limitation?"

11. Final report

As issued: "more clearly why we chose t5 instead of other options. what is advantage. how to explain. as per requirements make a pdf report for the project. Make sure to include analysis section consisting of comparison with existing models for this, Also, have a prompts issued section that contains the prompts given from our side to the agent. Refine these prompts, make them better."

Refined: "Write a clear, viva-ready explanation of why T5 was chosen and what its advantages are. Then produce the final report as a PDF that follows the assignment's section list, with an analysis section comparing our models against published question-generation systems on SQuAD, and a section listing the prompts we issued to the AI agent together with improved versions. Save the PDF in the repository and push it."

Observations. The most effective prompts specified the context (course, team, deadline and hardware), the deliverable and the acceptance criterion in a single message. Short follow-up questions proved to be the appropriate means of understanding the generated system before the viva. Instructing the agent to state explicitly what had not been attempted, and to report measured values only, kept the report accurate.

References

1. Rajpurkar, P., Zhang, J., Lopyrev, K. and Liang, P. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. Proceedings of EMNLP 2016.
2. Heilman, M. and Smith, N. A. (2010). Good Question! Statistical Ranking for Question Generation. Proceedings of NAACL-HLT 2010.
3. Du, X., Shao, J. and Cardie, C. (2017). Learning to Ask: Neural Question Generation for Reading Comprehension. Proceedings of ACL 2017.
4. Dong, L., Yang, N., Wang, W., Wei, F., Liu, X., Wang, Y., Gao, J., Zhou, M. and Hon, H.-W. (2019). Unified Language Model Pre-training for Natural Language Understanding and Generation. Advances in Neural Information Processing Systems 32.
5. Qi, W., Yan, Y., Gong, Y., Liu, D., Duan, N., Chen, J., Zhang, R. and Zhou, M. (2020). ProphetNet: Predicting Future N-gram for Sequence-to-Sequence Pre-training. Findings of EMNLP 2020.
6. Ushio, A., Alva-Manchego, F. and Camacho-Collados, J. (2022). Generative Language Models for Paragraph-Level Question Generation. Proceedings of EMNLP 2022. Model cards: huggingface.co/lmqg.
7. Alberti, C., Andor, D., Pitler, E., Devlin, J. and Collins, M. (2019). Synthetic QA Corpora Generation with Roundtrip Consistency. Proceedings of ACL 2019.
8. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W. and Liu, P. J. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. Journal of Machine Learning Research 21(140), 1–67.
9. Sanh, V., Debut, L., Chaumond, J. and Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108.
10. Zhao, Y., Ni, X., Ding, Y. and Ke, Q. (2018). Paragraph-level Neural Question Generation with Maxout Pointer and Gated Self-attention Networks. Proceedings of EMNLP 2018.

11. Honnibal, M., Montani, I., Van Landeghem, S. and Boyd, A. (2020). spaCy: Industrial-strength Natural Language Processing in Python. Software, spacy.io.
12. Wolf, T. et al. (2020). Transformers: State-of-the-Art Natural Language Processing. Proceedings of EMNLP 2020: System Demonstrations.